

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import math
```

First Five Rows

```
df = pd.read_csv(r"E:\Ekram Anlyst File\DATA SET\
ecommerce_eda_realistic_10000_rows.csv")
df.head()
```

	Order_ID	Order_Date	Customer_ID	Customer_Name	Age
Gender \					
0	ORD108776	2024-01-16 18:06:52	CUST11696	Customer_00646	25.0
Female					
1	ORD102199	2024-07-11 08:55:37	CUST14409	Customer_03442	25.0
Female					
2	ORD109596	2025-01-04 00:22:03	CUST11719	Customer_01222	53.0
Male					
3	ORD108490	2024-01-19 15:48:39	CUST11816	Customer_03341	41.0
Male					
4	ORD103314	2024-08-19 18:07:13	CUST13777	Customer_00800	35.0
Female					

	Customer_Segment	City	State	Region	...
Payment_Method \					
0	Consumer	Hyderabad	Telangana	West	...
Card					
1	Consumer	Mumbai	Maharashtra	South	...
UPI					
2	Corporate	Lucknow	Uttar Pradesh	West	...
Card					
3	Home Office	Mumbai	Maharashtra	West	...
Card					
4	Consumer	Chennai	Tamil Nadu	South	...
Delivery					

	Quantity	Unit_Price	Discount_Pct	Sales	Cost	Profit
Order_Status \						
0	2	723.27	5.0	1374.21	1090.44	283.77
Returned						
1	1	3935.13	5.0	3738.37	2435.14	1303.23
Cancelled						
2	1	1001.13	10.0	901.02	506.33	394.69
Delivered						
3	1	1180.93	30.0	826.65	524.94	301.71

Cancelled						
4	1	2051.33	0.0	2051.33	1295.93	755.40
Pending						

	Rating	Delivery_Days
0	5.0	7.0
1	5.0	6.0
2	4.0	4.0
3	5.0	4.0
4	4.0	6.0

[5 rows x 23 columns]

Last Five Rows

df.tail(5)

	Order_ID	Order_Date	Customer_ID	Customer_Name			
Age \							
10045	ORD105734	2024-11-04 18:48:27	CUST13520	Customer_02354			
63.0							
10046	ORD105191	2024-11-27 23:12:48	CUST13277	Customer_02437			
62.0							
10047	ORD105390	2024-02-23 08:35:46	CUST14048	Customer_02594			
64.0							
10048	ORD100860	2024-01-06 04:32:25	CUST12515	Customer_02045			
27.0							
10049	ORD107270	2025-04-14 21:52:39	CUST12429	Customer_00899			
28.0							
	Gender	Customer_Segment	City	State	Region	...	\
10045	Female	Home Office	Hyderabad	Telangana	East	...	
10046	Male	Corporate	Bengaluru	Karnataka	North	...	
10047	Male	Consumer	Hyderabad	Telangana	South	...	
10048	Male	Consumer	Delhi	Delhi	North	...	
10049	Female	Consumer	Delhi	Delhi	East	...	
	Payment_Method	Quantity	Unit_Price	Discount_Pct	Sales		
Cost \							
10045	Net Banking	1	1049.44	0.0	1049.44		
753.47							
10046	Credit Card	2	572.07	25.0	858.10		
698.17							
10047	Debit Card	1	1523.53	10.0	1371.18		
773.64							
10048	Cash on Delivery	4	866.20	5.0	3291.56		
2554.60							
10049	UPI	5	763.92	25.0	2864.70		

2335.23

	Profit	Order_Status	Rating	Delivery_Days
10045	295.97	Cancelled	2.0	7.0
10046	159.93	Cancelled	1.0	10.0
10047	597.54	Delivered	4.0	6.0
10048	736.96	Pending	2.0	8.0
10049	529.47	Delivered	4.0	4.0

[5 rows x 23 columns]

Duplicate Rows

```
df.duplicated().any()
```

```
np.True_
```

```
df.duplicated().sum()
```

```
np.int64(50)
```

```
df[df.duplicated(keep=False)]
```

	Order_ID	Order_Date	Customer_ID	Customer_Name	Age
\					
44	ORD102020	2025-12-06 04:16:44	CUST10561	Customer_03022	49.0
159	ORD104712	2025-10-22 05:45:18	CUST13875	Customer_01994	63.0
172	ORD100623	2024-06-24 20:03:37	CUST10387	Customer_01428	53.0
236	ORD103225	2025-10-03 22:02:04	CUST14715	Customer_01351	55.0
282	ORD103297	2025-03-27 18:05:25	CUST12685	Customer_02013	18.0
...	...	...	...	...	...
9539	ORD103880	2025-10-06 12:23:20	CUST10441	Customer_02298	45.0
9570	ORD102967	2025-02-25 17:10:09	CUST10283	Customer_00538	39.0
9736	ORD108921	2024-04-28 01:52:41	CUST10432	Customer_01725	35.0
9776	ORD101015	2024-12-27 04:20:52	CUST14964	Customer_01696	57.0
9902	ORD102551	2024-04-18 14:47:31	CUST13978	Customer_03415	53.0

	Gender	Customer_Segment	City	State	Region	...	\
44	Male	Consumer	Delhi	Delhi	North	...	

159	Female	Consumer	Lucknow	Uttar Pradesh	South	...
172	Female	Corporate	Mumbai	Maharashtra	North	...
236	Male	Consumer	Bengaluru	Karnataka	South	...
282	Female	Corporate	Delhi	Delhi	West	...
...	...	...	...	...	...	...
9539	Male	Consumer	Mumbai	Maharashtra	West	...
9570	Male	Consumer	Delhi	Delhi	North	...
9736	Male	Corporate	Hyderabad	Telangana	West	...
9776	Male	Consumer	Mumbai	Maharashtra	East	...
9902	Female	Consumer	Jaipur	Rajasthan	North	...

	Payment_Method	Quantity	Unit_Price	Discount_Pct	Sales
Cost \					
44	UPI	2	1273.72	10.0	2292.70
1314.63					
159	Wallet	1	231.46	15.0	196.74
146.61					
172	Cash on Delivery	1	802.09	25.0	601.57
381.70					
236	Credit Card	2	3454.06	5.0	6562.71
5171.07					
282	UPI	3	618.99	25.0	1392.73
999.81					
...	...	...	...	...	...
...					
9539	UPI	2	700.76	5.0	1331.44
732.52					
9570	Debit Card	1	791.89	10.0	712.70
512.39					
9736	UPI	3	555.43	15.0	1416.35
1056.30					
9776	Credit Card	2	3323.86	10.0	5982.95
4580.95					
9902	UPI	4	1942.11	15.0	6603.17
5373.67					

	Profit	Order_Status	Rating	Delivery_Days
44	978.07	Pending	3.0	2.0
159	50.13	Delivered	4.0	2.0
172	219.87	Delivered	4.0	4.0
236	1391.64	Delivered	5.0	4.0
282	392.92	Delivered	3.0	4.0
...	...	...	...	...
9539	598.92	Returned	3.0	8.0
9570	200.31	Delivered	5.0	10.0
9736	360.05	Returned	4.0	3.0
9776	1402.00	Returned	5.0	4.0
9902	1229.50	Delivered	5.0	3.0

[100 rows x 23 columns]

Remove Duplicates

```
df = df.drop_duplicates()
```

Check Duplicate Row

```
df.duplicated().any()
```

```
np.False_
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

Check Unique Values

```
df.nunique()
```

Order_ID	10000
Order_Date	9998
Customer_ID	4336
Customer_Name	3309
Age	47
Gender	2
Customer_Segment	3
City	13
State	9
Region	4
Category	7
Sub_Category	35
Sales_Channel	4
Payment_Method	6
Quantity	5
Unit_Price	9763
Discount_Pct	7
Sales	9881
Cost	9828
Profit	9636
Order_Status	4
Rating	5
Delivery_Days	10
dtype:	int64

Understand the structure and quality of a DataFrame.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 10000 entries, 0 to 10049
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Order_ID              10000 non-null  object 
1   Order_Date            10000 non-null  object 
2   Customer_ID           10000 non-null  object 
3   Customer_Name         10000 non-null  object 
4   Age                   9850 non-null   float64
5   Gender                9900 non-null   object 
6   Customer_Segment      10000 non-null  object 
7   City                  9881 non-null   object 
8   State                 10000 non-null  object 
9   Region                10000 non-null  object 
10  Category              10000 non-null  object 
11  Sub_Category          10000 non-null  object 
12  Sales_Channel          10000 non-null  object 
13  Payment_Method        10000 non-null  object 
14  Quantity              10000 non-null  int64  
15  Unit_Price            10000 non-null  float64
16  Discount_Pct          9900 non-null   float64
17  Sales                 10000 non-null  float64
18  Cost                  10000 non-null  float64
19  Profit                10000 non-null  float64
20  Order_Status          10000 non-null  object 
21  Rating                9800 non-null   float64
22  Delivery_Days         9850 non-null   float64
dtypes: float64(8), int64(1), object(14)
memory usage: 1.8+ MB
```

Quick statistical summary of Data

```
df.describe()
```

	Age	Quantity	Unit_Price	Discount_Pct
Sales \				
count	9850.000000	10000.000000	10000.000000	9900.000000
10000.000000				
mean	41.038376	1.914300	1356.188532	11.423232
2320.534449				
std	13.552324	1.065115	986.208659	8.490153

2380.674130				
min	18.000000	1.000000	150.000000	0.000000
127.500000				
25%	29.000000	1.000000	705.447500	5.000000
888.367500				
50%	41.000000	2.000000	1096.210000	10.000000
1582.190000				
75%	53.000000	2.000000	1697.772500	15.000000
2888.140000				
max	64.000000	5.000000	17930.480000	30.000000
44745.120000				

	Cost	Profit	Rating	Delivery_Days
count	10000.000000	10000.000000	9800.000000	9850.000000
mean	1589.610359	730.924090	3.905408	5.529949
std	1654.385056	788.298165	1.068398	2.857741
min	72.440000	30.070000	1.000000	1.000000
25%	605.065000	264.170000	3.000000	3.000000
50%	1071.720000	488.515000	4.000000	6.000000
75%	1973.420000	893.355000	5.000000	8.000000
max	30193.060000	14552.060000	5.000000	10.000000

Rows and Columns

```
df.shape
(10000, 23)
df.shape[0]
10000
df.shape[1]
23
```

Checking Missing Vlaues of Each Columns

```
df.isnull().sum()
Order_ID      0
Order_Date    0
Customer_ID   0
Customer_Name 0
Age           150
Gender        100
Customer_Segment 0
```

```
City          119
State          0
Region         0
Category       0
Sub_Category  0
Sales_Channel  0
Payment_Method 0
Quantity       0
Unit_Price     0
Discount_Pct   100
Sales          0
Cost           0
Profit         0
Order_Status   0
Rating         200
Delivery_Days  150
dtype: int64
```

826 Blank rows are present

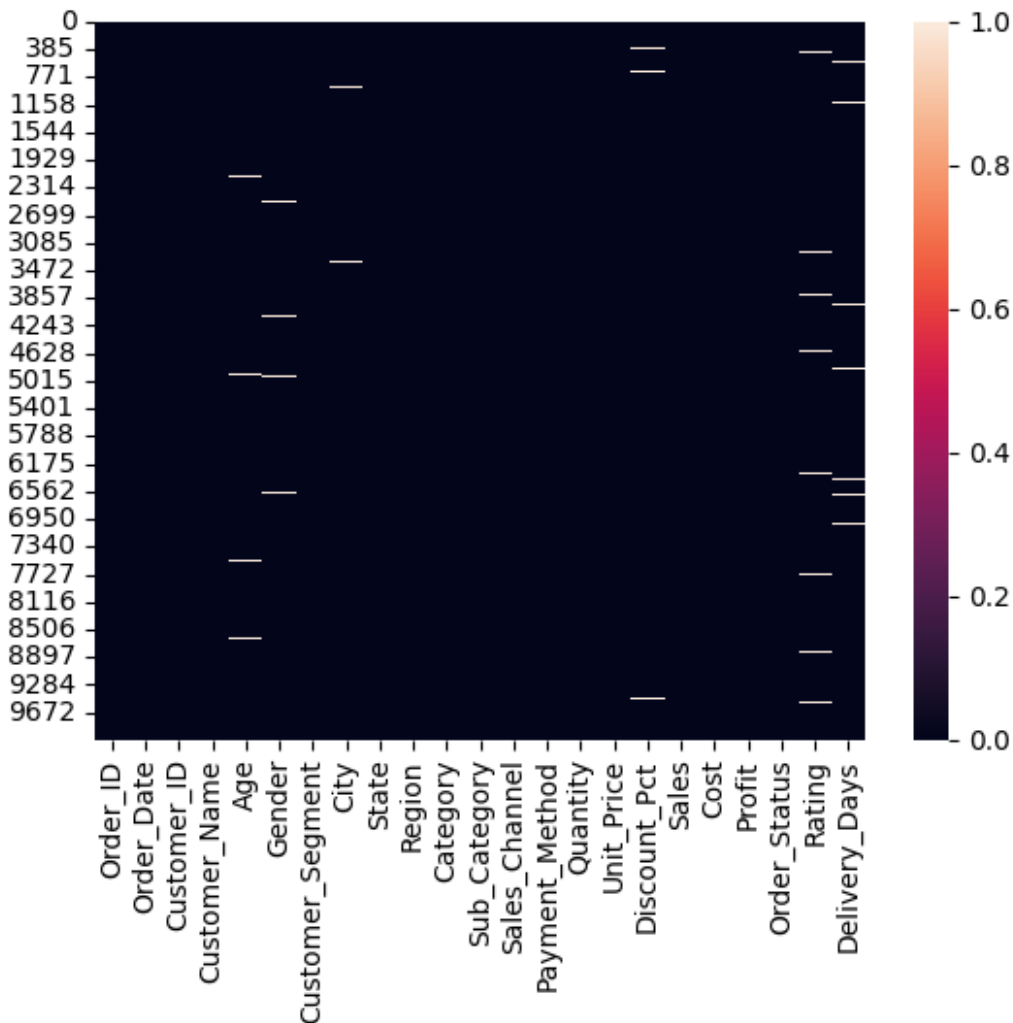
```
df.isnull().sum().sum()
np.int64(819)
```

Null Values In Percentage

```
(df.isnull().sum().sum()/df.shape[0])*100
np.float64(8.19)
```

Visually see missing values in DataFrame.

```
sns.heatmap(df.isnull())
plt.show()
```



Filling Method One by One Column Through
MEAN, MEDIAN and MODE

Filling Method Using FOR Loop in Multiple
Columns

```
for i in df.select_dtypes(include=["int", "float"]).columns:
    print(i)

Age
Quantity
Unit_Price
Discount_Pct
```

```

Sales
Cost
Profit
Rating
Delivery_Days

for i in df.select_dtypes(include=["int","float"]).columns:
    df[i].fillna(df[i].median(),inplace = True)

df =
df.astype({'Age':'int','Quantity':'int','Unit_Price':'int','Discount_P
ct':'int','Sales':'int','Cost':'int','Profit':'int','Rating':'int','De
livery_Days':'int'})

```

Fillna Method To Fill Missing Vlaues

```

df["Gender"] = df["Gender"].fillna("Unknown")

df["City"] = df["City"].fillna("Unknown")

```

Columns Name

```

df.columns

Index(['Order_ID', 'Order_Date', 'Customer_ID', 'Customer_Name',
'Age',
      'Gender', 'Customer_Segment', 'City', 'State', 'Region',
'Category',
      'Sub_Category', 'Sales_Channel', 'Payment_Method', 'Quantity',
      'Unit_Price', 'Discount_Pct', 'Sales', 'Cost', 'Profit',
'Order_Status',
      'Rating', 'Delivery_Days'],
      dtype='object')

```

Covert Columns Name In Lower Case

```

df.columns = df.columns.str.lower()

df.columns

Index(['order_id', 'order_date', 'customer_id', 'customer_name',
'age',
      'gender', 'customer_segment', 'city', 'state', 'region',
'category',
      'sub_category', 'sales_channel', 'payment_method', 'quantity',

```

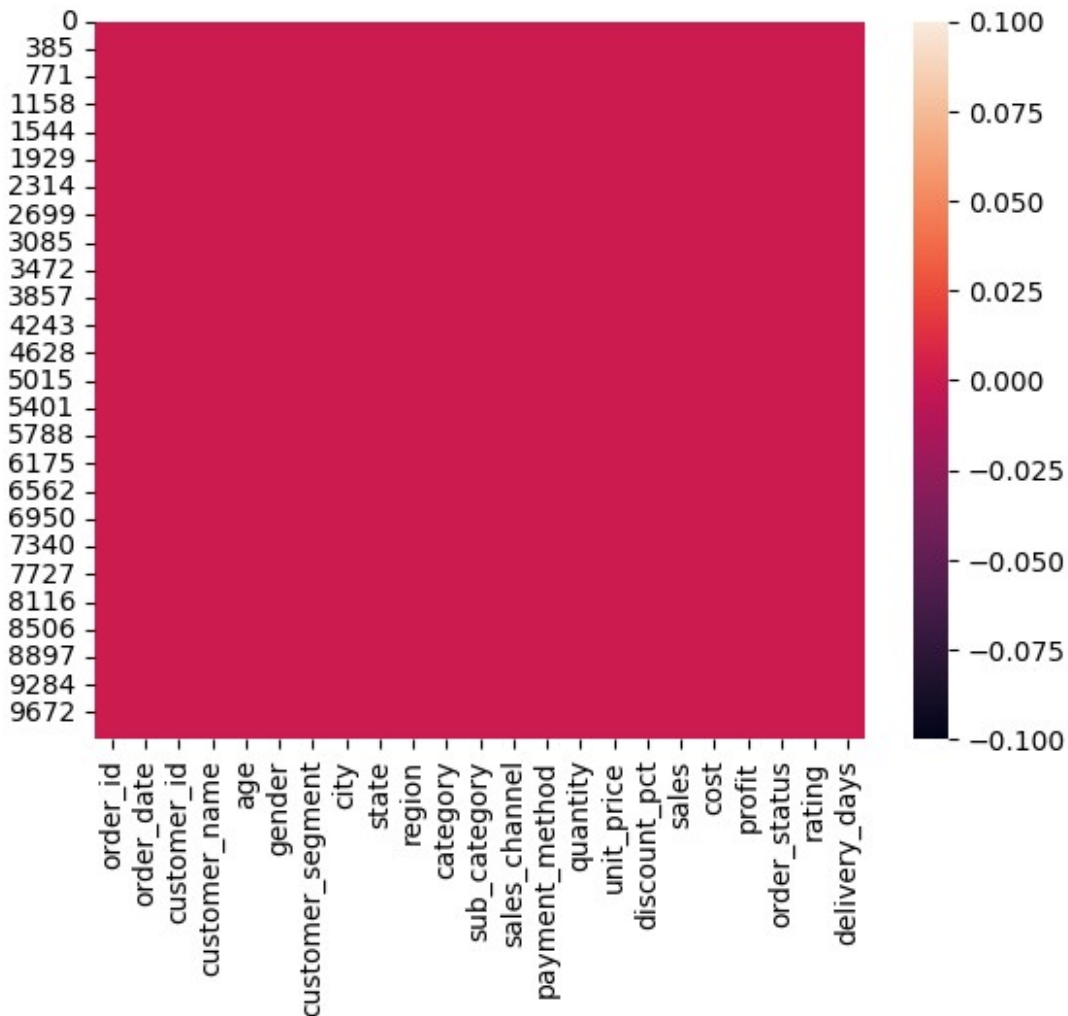
```
    'unit_price', 'discount_pct', 'sales', 'cost', 'profit',  
'order_status',  
    'rating', 'delivery_days'],  
    dtype='object')
```

Convert Datatypes

```
df[["age", "rating", "delivery_days"]] =  
df[["age", "rating", "delivery_days"]].astype("Int64")
```

Check Missing Values

```
df.isnull().sum().sum()  
  
np.int64(0)  
  
sns.heatmap(df.isnull())  
plt.show()
```



Convert Datatype and Extract YEAR, MONTH, DAY, MONTH_NAME, DAY_NAME

```
df["order_date"] = pd.to_datetime(df["order_date"])
df["year"] = df["order_date"].dt.year
df["month"] = df["order_date"].dt.month
df["month_name"] = df["order_date"].dt.month_name()
df["day"] = df["order_date"].dt.day
df["day_name"] = df["order_date"].dt.day_name()

df.columns

Index(['order_id', 'order_date', 'customer_id', 'customer_name',
      'age',
      'gender', 'customer_segment', 'city', 'state', 'region',
      'category',
```

```

        'sub_category', 'sales_channel', 'payment_method', 'quantity',
        'unit_price', 'discount_pct', 'sales', 'cost', 'profit',
'order_status',
        'rating', 'delivery_days', 'year', 'month', 'month_name',
'day',
        'day_name'],
        dtype='object')

```

Remove Extra Space

```
df["city"] = df["city"].str.strip()
```

Replace Entry

```
df["city"] = df["city"].replace('Hyderbad', 'Hyderabad')
```

Now Dataset Is Cleaned

```
df.head(5)
```

	order_id	order_date	customer_id	customer_name	age
gender \					
0	ORD108776	2024-01-16 18:06:52	CUST11696	Customer_00646	25
Female					
1	ORD102199	2024-07-11 08:55:37	CUST14409	Customer_03442	25
Female					
2	ORD109596	2025-01-04 00:22:03	CUST11719	Customer_01222	53
Male					
3	ORD108490	2024-01-19 15:48:39	CUST11816	Customer_03341	41
Male					
4	ORD103314	2024-08-19 18:07:13	CUST13777	Customer_00800	35
Female					

	customer_segment	city	state	region	...	cost	profit
\							
0	Consumer	Hyderabad	Telangana	West	...	1090	283
1	Consumer	Mumbai	Maharashtra	South	...	2435	1303
2	Corporate	Lucknow	Uttar Pradesh	West	...	506	394
3	Home Office	Mumbai	Maharashtra	West	...	524	301
4	Consumer	Chennai	Tamil Nadu	South	...	1295	755

	order_status	rating	delivery_days	year	month	month_name	day
0	Returned	5	7	2024	1	January	16
1	Cancelled	5	6	2024	7	July	11
2	Delivered	4	4	2025	1	January	4
3	Cancelled	5	4	2024	1	January	19
4	Pending	4	6	2024	8	August	19

[5 rows x 28 columns]

Save Clean Dataset into CSV format

```
# df.to_csv("E:\\Ekram Anlyst File\\DATA SET\\
Clean_Ecommerce_Data_EDA.csv",index=False)
```

```
df["state"].value_counts()
```

```
state
Maharashtra    2026
Telangana      1063
West Bengal    1040
Tamil Nadu     1006
Rajasthan      1005
Karnataka      993
Gujarat        986
Uttar Pradesh  949
Delhi          932
Name: count, dtype: int64
```

```
df["city"].value_counts()
```

```
city
Hyderabad    1054
Kolkata      1024
Mumbai       1010
Pune         998
Chennai      993
Jaipur       992
Bengaluru    985
Ahmedabad    973
Lucknow      931
Delhi        921
```

```

Unknown      119
Name: count, dtype: int64

df["gender"].value_counts()

gender
Male      5067
Female    4833
Unknown    100
Name: count, dtype: int64

df["state"].unique()

array(['Telangana', 'Maharashtra', 'Uttar Pradesh', 'Tamil Nadu',
      'Delhi',
      'Rajasthan', 'Gujarat', 'Karnataka', 'West Bengal'],
      dtype=object)

df["state"].nunique()

9

df[['unit_price', 'discount_pct', 'sales', 'cost', 'profit']]

   unit_price  discount_pct  sales  cost  profit
0         723             5   1374  1090    283
1        3935             5   3738  2435   1303
2        1001            10    901   506    394
3        1180            30    826   524    301
4        2051             0   2051  1295    755
...         ...          ...   ...   ...   ...
10045       1049             0   1049   753    295
10046        572            25    858   698    159
10047       1523            10   1371   773    597
10048        866             5   3291  2554    736
10049        763            25   2864  2335    529

[10000 rows x 5 columns]

```

Minimum Sales Of Each Product

```

print("Minimum Sales")
Minimum_Sales = df.groupby("category", as_index=False)
["sales"].min().sort_values("sales")
Minimum_Sales
#print(f"Rs {int(Minimum_Sales):,.2f}")

Minimum Sales

```

	category	sales
2	Clothing	127
4	Furniture	127
0	Beauty	137
5	Home & Kitchen	155
3	Electronics	156
1	Books	191
6	Sports	207

Maximum Sales of Each Category

```
print("Maximum Sales")
Maximum_Sales= df.groupby("category",as_index=False)["sales"].max()
Maximum_Sales
#print(f"Rs {int(Maximum_Sales):,.2f}")
```

Maximum Sales

	category	sales
0	Beauty	44745
1	Books	23222
2	Clothing	34068
3	Electronics	23873
4	Furniture	29369
5	Home & Kitchen	17819
6	Sports	19929

Total Sales

```
print("Total Sales")
Total_Sales = df["sales"].sum()
print(f"Rs {int(Total_Sales):,.2f}")
```

Total Sales
Rs 23,200,385.00

```
print("Total cost")
Total_cost = df["cost"].sum()
print(f"Rs {Total_cost:,.2f}")
```

Total cost
Rs 15,891,151.00

Total Profit

```
print("Total Profit")
Total_Profit = df["profit"].sum()
print(f"Rs {Total_Profit:,.2f}")
```

Total Profit
Rs 7,304,314.00

Average Sales Each Day

```
Average_Sales = df["sales"].mean()
print(f" Average Sales Rs {int(Average_Sales):,.2f}")
```

Average Sales Rs 2,320.00

Total Quantity

```
total_quantity = df["quantity"].sum()
print(f"Total Quantity {total_quantity}")
```

Total Quantity 19143

Average Quantity Sales Each Day

```
average_quantity = df["quantity"].mean()
print(f"Average Quantity: {math.ceil(average_quantity)}")
```

Average Quantity: 2

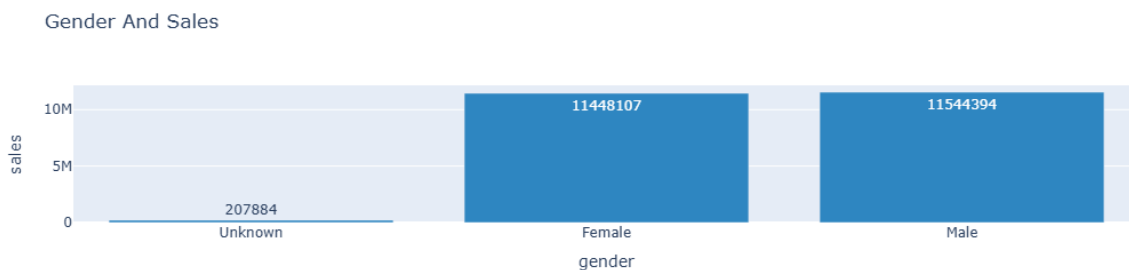
Gender Wise Sales

```
print("Gender Wise Sales")
Gender_wise_sales = df.groupby(['gender'],as_index=False)
["sales"].sum().sort_values("sales")
Gender_wise_sales
```

Gender Wise Sales

	gender	sales
2	Unknown	207884
0	Female	11448107
1	Male	11544394

```
fig = px.bar( Gender_wise_sales,
              x="gender",
              y="sales",
              title="Gender And Sales",
              text = "sales",
              color_discrete_sequence=["#2E86C1"]
            )
fig.update_layout(
    width=600,
    height=300
)
fig.show()
```

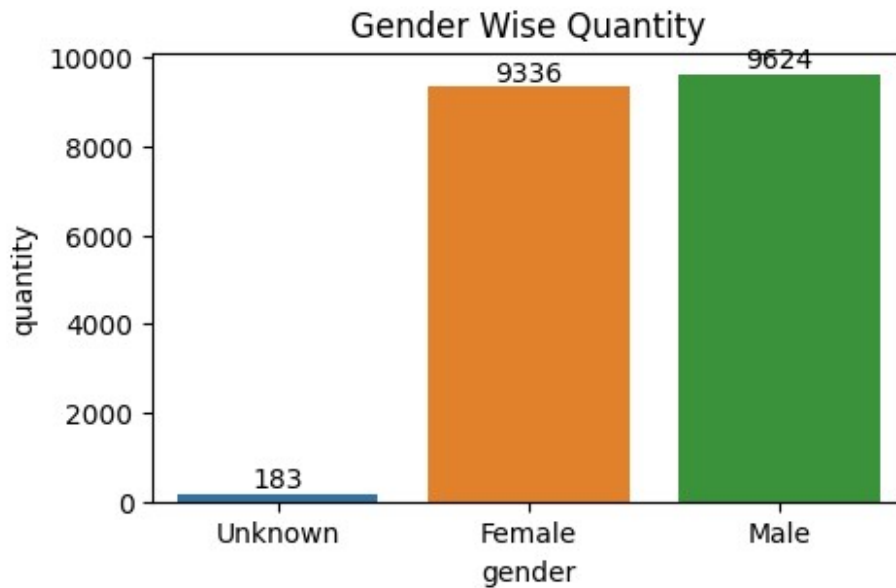


Gender wise Quantity Sale

```
Gender_wise_quantity = df.groupby(["gender"],as_index=False)
["quantity"].sum().sort_values("quantity")
Gender_wise_quantity
```

	gender	quantity
2	Unknown	183
0	Female	9336
1	Male	9624

```
plt.figure(figsize=(5,3))
gwq = sns.barplot(x = "gender", y =
"quantity",data=Gender_wise_quantity, hue="gender")
for x in gwq.containers:
    gwq.bar_label(x)
plt.title("Gender Wise Quantity")
plt.show()
```



Create Age Group

```
df["age_group"] = pd.cut(  
    df["age"],  
    bins=[0, 18, 30, 45, 60, 100],  
    labels=["0-18", "19-30", "31-45", "46-60", "60+"]  
)
```

Age Group By Sales Amount, Quantity and Profit

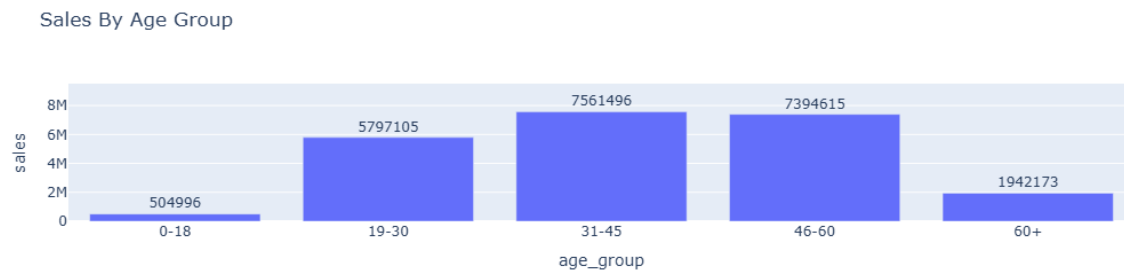
```
print("Sales By Age Group")  
age_group_sales = df.groupby(["age_group"], as_index=False)  
["sales"].sum().sort_values("age_group")  
age_group_sales
```

Sales By Age Group

```
C:\Users\computer\AppData\Local\Temp\ipykernel_3116\2835982866.py:2:  
FutureWarning: The default of observed=False is deprecated and will be  
changed to True in a future version of pandas. Pass observed=False to  
retain current behavior or observed=True to adopt the future default  
and silence this warning.  
age_group_sales = df.groupby(["age_group"], as_index=False)  
["sales"].sum().sort_values("age_group")
```

	age_group	sales
0	0-18	504996
1	19-30	5797105
2	31-45	7561496
3	46-60	7394615
4	60+	1942173

```
fig = px.bar(
    age_group_sales,
    x="age_group",
    y="sales",
    title="Sales By Age Group",
    text="sales"
)
fig.update_layout(
    width=600,
    height=300
)
fig.update_traces(texttemplate="%{text:.0f}", textposition="outside")
fig.show()
```



```
print("Quantity By Age Group")
age_group_quantity = df.groupby(["age_group"],as_index=False)
["quantity"].sum().sort_values("age_group")
age_group_quantity
```

Quantity By Age Group

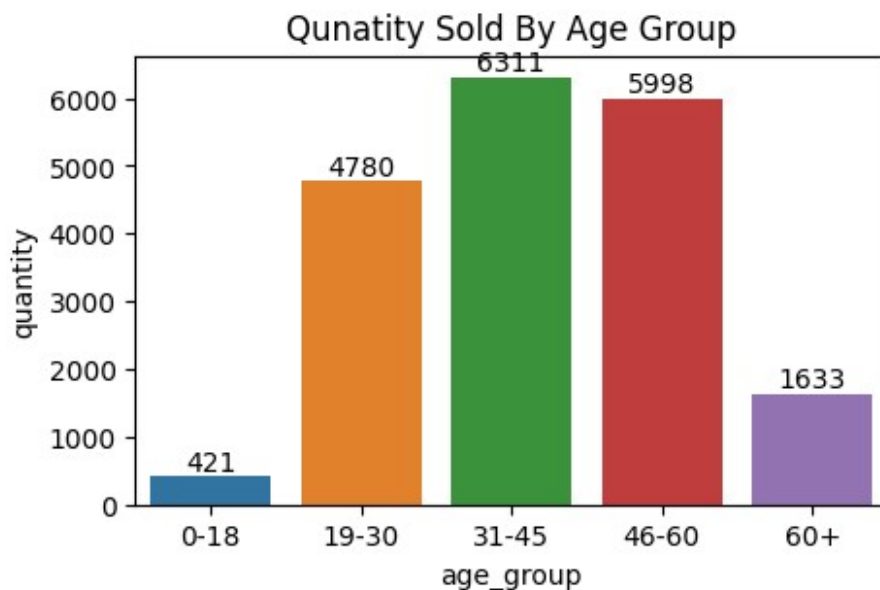
C:\Users\computer\AppData\Local\Temp\ipykernel_3116\245921341.py:2:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.

```
age_group_quantity = df.groupby(["age_group"],as_index=False)
["quantity"].sum().sort_values("age_group")
```

	age_group	quantity
0	0-18	421
1	19-30	4780

2	31-45	6311
3	46-60	5998
4	60+	1633

```
plt.figure(figsize=(5,3))
agq = sns.barplot(x="age_group",y="quantity",data=age_group_quantity,
hue="age_group")
for x in agq.containers:
    agq.bar_label(x)
plt.title("Qunatity Sold By Age Group")
plt.show()
```



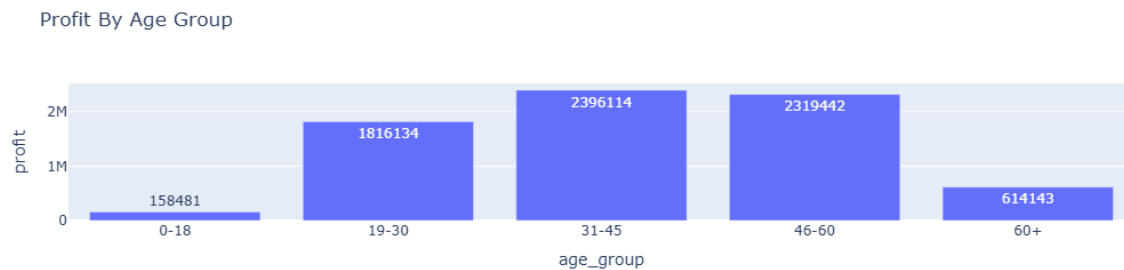
```
age_group_profit = df.groupby(["age_group"],as_index=False)
["profit"].sum().sort_values("age_group")
age_group_profit
```

C:\Users\computer\AppData\Local\Temp\ipykernel_3116\3348602631.py:1:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.

```
age_group_profit = df.groupby(["age_group"],as_index=False)
["profit"].sum().sort_values("age_group")
```

	age_group	profit
0	0-18	158481
1	19-30	1816134
2	31-45	2396114
3	46-60	2319442
4	60+	614143

```
fig = px.bar(age_group_profit,x="age_group",y="profit",title="Profit
By Age Group",text="profit")
fig.update_layout(
    width=600,
    height=300
)
fig.show()
```



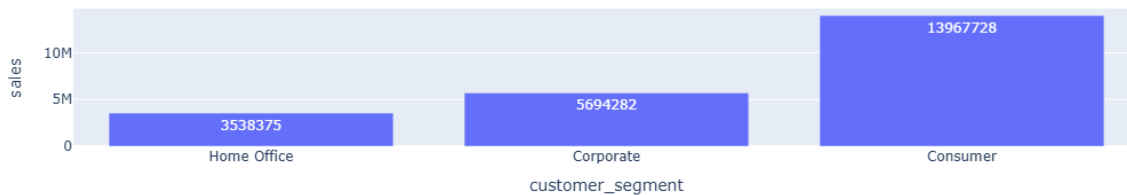
Customer Segment Wise Sales , Quantity and Profit

```
customer_segment_sales =
df.groupby(["customer_segment"],as_index=False)
["sales"].sum().sort_values("sales")
customer_segment_sales
```

	customer_segment	sales
2	Home Office	3538375
1	Corporate	5694282
0	Consumer	13967728

```
fig =
px.bar(customer_segment_sales,x="customer_segment",y="sales",title="Sa
les By Customer_Segment",text="sales")
fig.update_layout(
    width=600,
    height=300
)
fig.show()
```

Sales By Customer_Segment

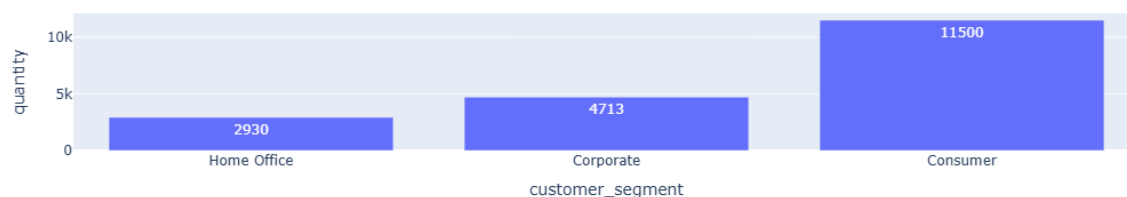


```
customer_segment_qnt = df.groupby(["customer_segment"],as_index=False)
["quantity"].sum().sort_values("quantity")
customer_segment_qnt
```

	customer_segment	quantity
2	Home Office	2930
1	Corporate	4713
0	Consumer	11500

```
fig =
px.bar(customer_segment_qnt,x="customer_segment",y="quantity",title="Q
uantity Sold By Customer_Segment",text="quantity")
fig.update_layout(
    width=550,
    height=300
)
fig.show()
```

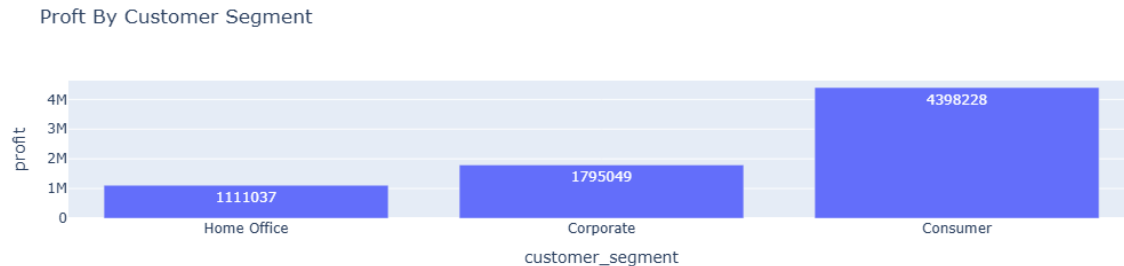
Quantity Sold By Customer_Segment



```
customer_segment_profit =
df.groupby(["customer_segment"],as_index=False)
["profit"].sum().sort_values("profit")
customer_segment_profit
```

	customer_segment	profit
2	Home Office	1111037
1	Corporate	1795049
0	Consumer	4398228

```
fig =
px.bar(customer_segment_profit,x="customer_segment",y="profit",title="
Profit By Customer Segment",text="profit")
fig.update_layout(
    width=550,
    height=300
)
fig.show()
```



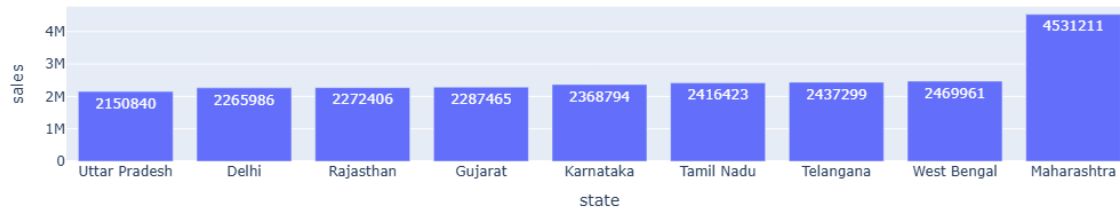
State wise sales Amount, Quantity and Profit

```
state_sales = df.groupby(["state"],as_index=False)
["sales"].sum().sort_values("sales")
state_sales
```

	state	sales
7	Uttar Pradesh	2150840
0	Delhi	2265986
4	Rajasthan	2272406
1	Gujarat	2287465
2	Karnataka	2368794
5	Tamil Nadu	2416423
6	Telangana	2437299
8	West Bengal	2469961
3	Maharashtra	4531211

```
fig = px.bar(state_sales,x="state",y="sales",title="State Wise Sales
Amount",text="sales")
fig.update_layout(
    width=1100,    # reduce width
    height=300     # graph height
)
fig.show()
```

State Wise Sales Amount

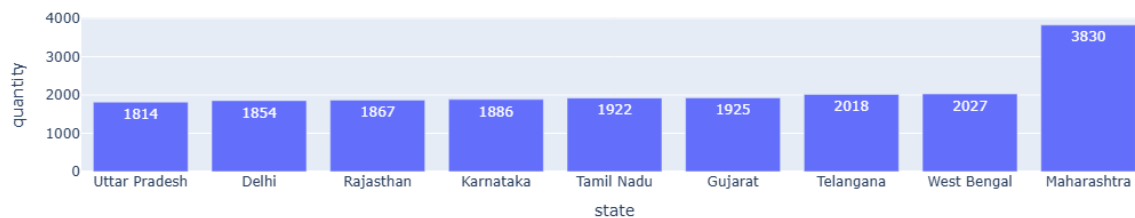


```
state_quantity = df.groupby(["state"],as_index=False)
["quantity"].sum().sort_values("quantity")
state_quantity
```

	state	quantity
7	Uttar Pradesh	1814
0	Delhi	1854
4	Rajasthan	1867
2	Karnataka	1886
5	Tamil Nadu	1922
1	Gujarat	1925
6	Telangana	2018
8	West Bengal	2027
3	Maharashtra	3830

```
fig = px.bar(state_quantity,x="state",y="quantity",title="State Wise
Quantity Sold",text="quantity")
fig.update_layout(
    width=1100,    # reduce width
    height=300     # graph height
)
fig.show()
```

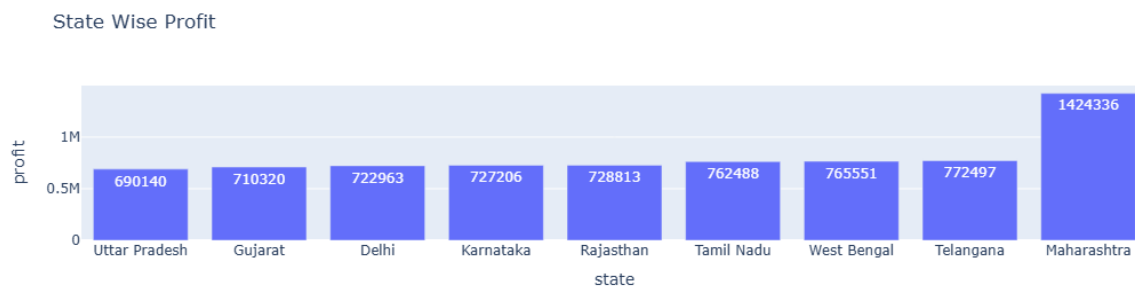
State Wise Quantity Sold



```
state_profit = df.groupby(["state"],as_index=False)
["profit"].sum().sort_values("profit")
state_profit
```

	state	profit
7	Uttar Pradesh	690140
1	Gujarat	710320
0	Delhi	722963
2	Karnataka	727206
4	Rajasthan	728813
5	Tamil Nadu	762488
8	West Bengal	765551
6	Telangana	772497
3	Maharashtra	1424336

```
fig = px.bar(state_profit,x="state",y="profit",title="State Wise Profit",text="profit")
fig.update_layout(
    width=1150,
    height=300
)
fig.show()
```

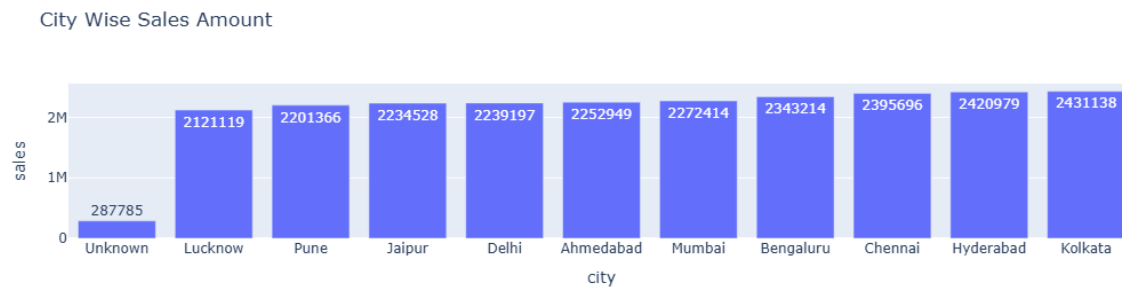


City wise Sales, Quantity and Profit

```
city_sales = df.groupby(["city"],as_index=False)
["sales"].sum().sort_values("sales")
city_sales
```

	city	sales
10	Unknown	287785
7	Lucknow	2121119
9	Pune	2201366
5	Jaipur	2234528
3	Delhi	2239197
0	Ahmedabad	2252949
8	Mumbai	2272414
1	Bengaluru	2343214
2	Chennai	2395696
4	Hyderabad	2420979
6	Kolkata	2431138

```
fig = px.bar(city_sales,x="city",y="sales",title="City Wise Sales Amount",text="sales")
fig.update_layout(
    width=1100,    # reduce width
    height=300     # graph height
)
fig.show()
```

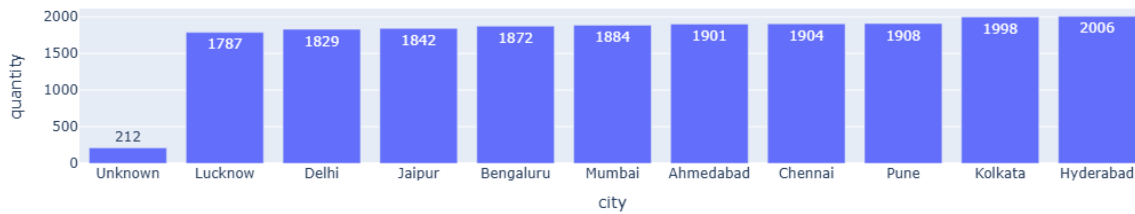


```
city_quantity = df.groupby(["city"],as_index=False)
["quantity"].sum().sort_values("quantity")
city_quantity
```

	city	quantity
10	Unknown	212
7	Lucknow	1787
3	Delhi	1829
5	Jaipur	1842
1	Bengaluru	1872
8	Mumbai	1884
0	Ahmedabad	1901
2	Chennai	1904
9	Pune	1908
6	Kolkata	1998
4	Hyderabad	2006

```
fig = px.bar(city_quantity,x="city",y="quantity",title="City Wise Quantity Sold",text="quantity")
fig.update_layout(
    width=1100,    # reduce width
    height=300     # graph height
)
fig.show()
```

City Wise Quantity Sold



```
print("City wise Profit")
city_profit = df.groupby(["city"],as_index=False)
["profit"].sum().sort_values("profit")
city_profit
```

City wise Profit

	city	profit
10	Unknown	86293
7	Lucknow	681070
9	Pune	698728
0	Ahmedabad	699388
8	Mumbai	710628
3	Delhi	714827
5	Jaipur	716819
1	Bengaluru	719030
6	Kolkata	753791
2	Chennai	755721
4	Hyderabad	768019

```
fig = px.bar(city_profit,x="city",y="profit",title="City Wise Profit",text="profit")
fig.update_layout(
    width=1100,    # reduce width
    height=300     # graph height
)
fig.show()
```

City Wise Profit



Region wise Sales and Quantity and Profit

```
region_sales = df.groupby(["region"],as_index=False)
["sales"].sum().sort_values("sales")
region_sales
```

	region	sales
1	North	5719548
0	East	5779095
2	South	5838535
3	West	5863207

```
fig = px.bar(region_sales,x="region",y="sales",title="Region Wise Sales",text="sales")
fig.update_layout(
    width=600,    # reduce width
    height=300    # graph height
)
fig.show()
```

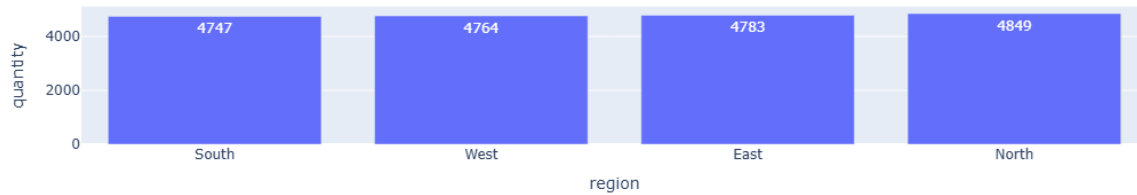


```
region_quantity = df.groupby(["region"],as_index=False)
["quantity"].sum().sort_values("quantity")
region_quantity
```

	region	quantity
2	South	4747
3	West	4764
0	East	4783
1	North	4849

```
fig = px.bar(region_quantity,x="region",y="quantity",title="Region Wise Quantity Sold",text="quantity")
fig.update_layout(
    width=600,    # reduce width
    height=300    # graph height
)
fig.show()
```

Region Wise Quantity Sold

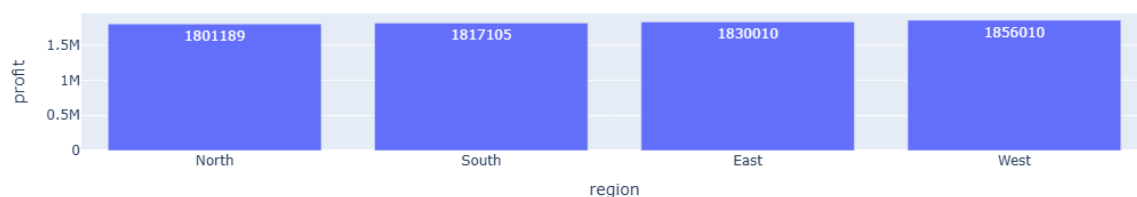


```
region_profit = df.groupby(["region"],as_index=False)
["profit"].sum().sort_values("profit")
region_profit
```

	region	profit
1	North	1801189
2	South	1817105
0	East	1830010
3	West	1856010

```
fig = px.bar(region_profit,x="region",y="profit",title="Region Wise Profit",text="profit")
fig.update_layout(
    width=600, # reduce width
    height=300 # graph height
)
fig.show()
```

Region Wise Profit



Category wise Sales Amount , Quantity and Profit

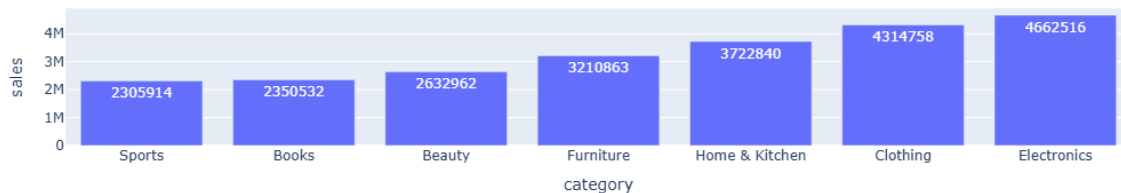
```
print("Category wise Sales Amount")
category_sales = df.groupby("category",as_index=False)
["sales"].sum().sort_values("sales",ascending = True)
category_sales
```

Category wise Sales Amount

	category	sales
6	Sports	2305914
1	Books	2350532
0	Beauty	2632962
4	Furniture	3210863
5	Home & Kitchen	3722840
2	Clothing	4314758
3	Electronics	4662516

```
fig = px.bar(category_sales,x="category",y="sales",title="Sales Amount  
By Category",text="sales")  
fig.update_layout(  
    width=800,  
    height=300  
)  
fig.show()
```

Sales Amount By Category



```
print("Category wise Quantity")  
category_quantity = df.groupby("category",as_index=False)  
["quantity"].sum().sort_values("quantity",ascending = True)  
category_quantity
```

Category wise Quantity

	category	quantity
1	Books	1901
6	Sports	1935
0	Beauty	2211
4	Furniture	2697
5	Home & Kitchen	3005
2	Clothing	3599
3	Electronics	3795

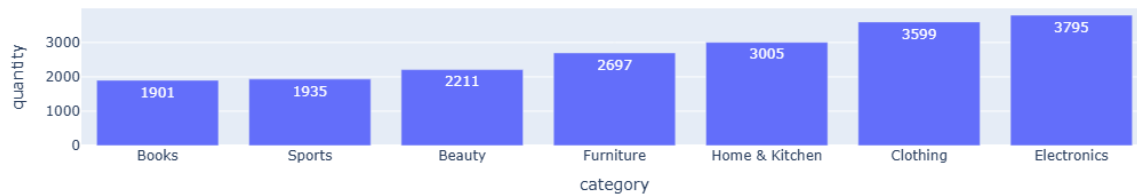
```
fig =  
px.bar(category_quantity,x="category",y="quantity",title="Category  
Wise Quantity",text="quantity")  
fig.update_layout()
```

```

        width=800,
        height=300
    )
    fig.show()

```

Category Wise Quantity



```

print("Category wise Profit")
category_profit = df.groupby("category",as_index=False)
["profit"].sum().sort_values("profit",ascending = True)
category_profit

```

Category wise Profit

	category	profit
6	Sports	717121
1	Books	729177
0	Beauty	827567
4	Furniture	1024360
5	Home & Kitchen	1169393
2	Clothing	1348666
3	Electronics	1488030

```

fig = px.bar(category_profit,x="category",y="profit",title="Category
Wise Profit",text="profit")
fig.update_layout(
    width=800,
    height=300
)
fig.show()

```

Category Wise Profit



Sub Category wise Sales ,Quantity and Profit

```
print("sub_category wise Sales")
sub_category_sales = df.groupby("sub_category",as_index=False)
["sales"].sum().sort_values("sales",ascending = False)
sub_category_sales
```

sub_category wise Sales

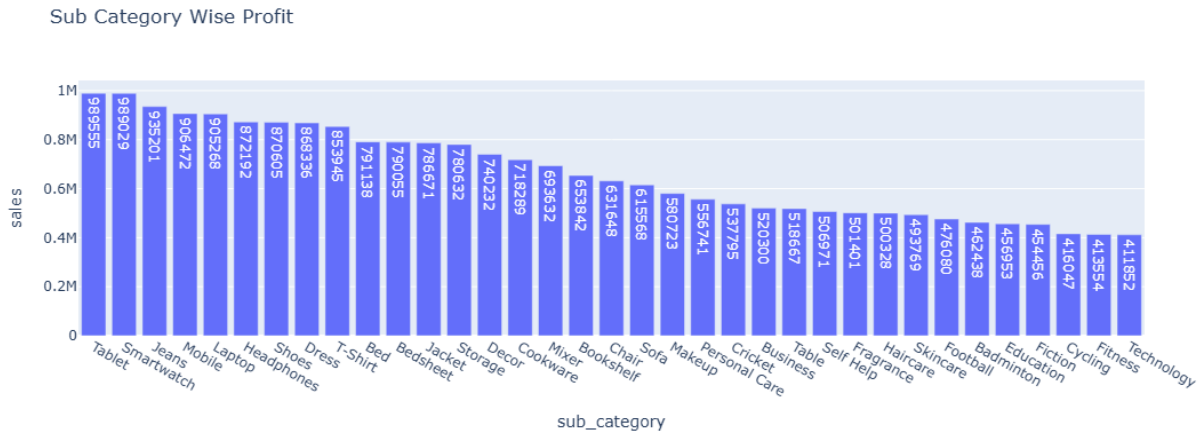
	sub_category	sales
33	Tablet	989555
28	Smartwatch	989029
19	Jeans	935201
23	Mobile	906472
20	Laptop	905268
17	Headphones	872192
26	Shoes	870605
10	Dress	868336
31	T-Shirt	853945
1	Bed	791138
2	Bedsheet	790055
18	Jacket	786671
30	Storage	780632
9	Decor	740232
6	Cookware	718289
22	Mixer	693632
3	Bookshelf	653842
5	Chair	631648
29	Sofa	615568
21	Makeup	580723
24	Personal Care	556741
7	Cricket	537795
4	Business	520300
32	Table	518667
25	Self Help	506971
15	Fragrance	501401
16	Haircare	500328
27	Skincare	493769
14	Football	476080
0	Badminton	462438
11	Education	456953
12	Fiction	454456
8	Cycling	416047
13	Fitness	413554
34	Technology	411852

```
fig = px.bar(sub_category_sales,x="sub_category",y="sales",title="Sub  
Category Wise Profit",text="sales")
fig.update_layout(
    width=1100,
```

```

        height=400
    )
    fig.show()

```



Sub Category wise Sales, Quantity and Profit

```

print("sub_category Wise Quantity")
sub_category_qnt = df.groupby("sub_category",as_index=False)
["quantity"].sum().sort_values("quantity",ascending = False)
sub_category_qnt

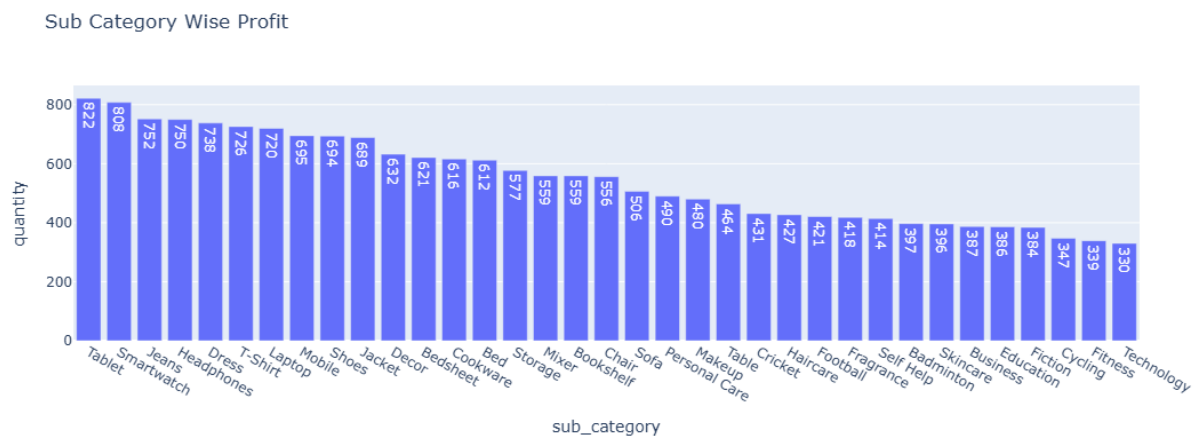
```

sub_category Wise Quantity

	sub_category	quantity
33	Tablet	822
28	Smartwatch	808
19	Jeans	752
17	Headphones	750
10	Dress	738
31	T-Shirt	726
20	Laptop	720
23	Mobile	695
26	Shoes	694
18	Jacket	689
9	Decor	632
2	Bedsheet	621
6	Cookware	616
1	Bed	612
30	Storage	577
22	Mixer	559
3	Bookshelf	559
5	Chair	556
29	Sofa	506

24	Personal Care	490
21	Makeup	480
32	Table	464
7	Cricket	431
16	Haircare	427
14	Football	421
15	Fragrance	418
25	Self Help	414
0	Badminton	397
27	Skincare	396
4	Business	387
11	Education	386
12	Fiction	384
8	Cycling	347
13	Fitness	339
34	Technology	330

```
fig = px.bar(sub_category_qnt,x="sub_category",y="quantity",title="Sub
Category Wise Profit",text="quantity")
fig.update_layout(
    width=1100,
    height=400
)
fig.show()
```



```
print("sub_category Wise Profit")
sub_category_profit = df.groupby("sub_category",as_index=False)
["profit"].sum().sort_values("profit",ascending = False)
sub_category_profit
```

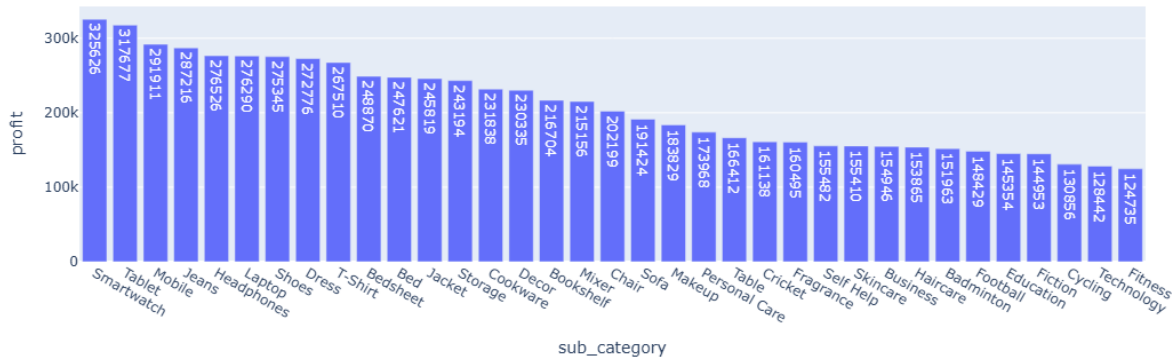
sub_category Wise Profit

	sub_category	profit
28	Smartwatch	325626
33	Tablet	317677

23	Mobile	291911
19	Jeans	287216
17	Headphones	276526
20	Laptop	276290
26	Shoes	275345
10	Dress	272776
31	T-Shirt	267510
2	Bedsheet	248870
1	Bed	247621
18	Jacket	245819
30	Storage	243194
6	Cookware	231838
9	Decor	230335
3	Bookshelf	216704
22	Mixer	215156
5	Chair	202199
29	Sofa	191424
21	Makeup	183829
24	Personal Care	173968
32	Table	166412
7	Cricket	161138
15	Fragrance	160495
25	Self Help	155482
27	Skincare	155410
4	Business	154946
16	Haircare	153865
0	Badminton	151963
14	Football	148429
11	Education	145354
12	Fiction	144953
8	Cycling	130856
34	Technology	128442
13	Fitness	124735

```
fig =
px.bar(sub_category_profit,x="sub_category",y="profit",title="Sub
Category Wise Profit",text="profit")
fig.update_layout(
    width=1100,
    height=400
)
fig.show()
```

Sub Category Wise Profit



Sales Channel Wise Sales, Quantity and Profit

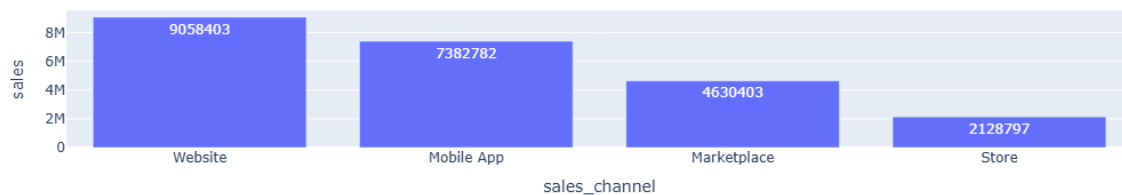
```
print("sales chanel wise Total Sales")
sc_sales = df.groupby("sales_channel",as_index=False)
["sales"].sum().sort_values("sales",ascending = False)
sc_sales
```

sales chanel wise Total Sales

	sales_channel	sales
3	Website	9058403
1	Mobile App	7382782
0	Marketplace	4630403
2	Store	2128797

```
fig = px.bar(sc_sales,x="sales_channel",y="sales",title="Channel Wise Sales",text="sales")
fig.update_layout(
    width=600,
    height=300
)
fig.show()
```

Channel Wise Sales



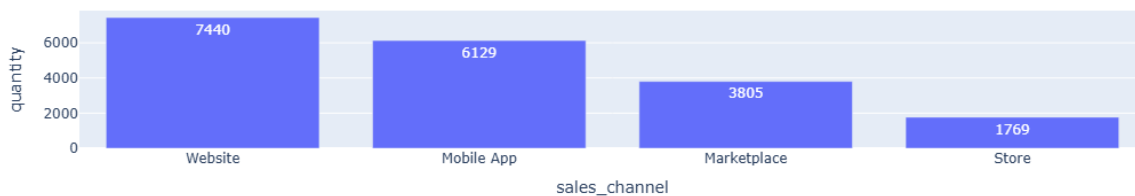
```
print("sales chanel wise Total Quantity")
sc_qnt = df.groupby("sales_channel",as_index=False)
["quantity"].sum().sort_values("quantity",ascending = False)
sc_qnt
```

sales chanel wise Total Quantity

	sales_channel	quantity
3	Website	7440
1	Mobile App	6129
0	Marketplace	3805
2	Store	1769

```
fig = px.bar(sc_qnt,x="sales_channel",y="quantity",title="Channel Wise
Quantity Sold",text="quantity")
fig.update_layout(
    width=600,
    height=300
)
fig.show()
```

Channel Wise Quantity Sold



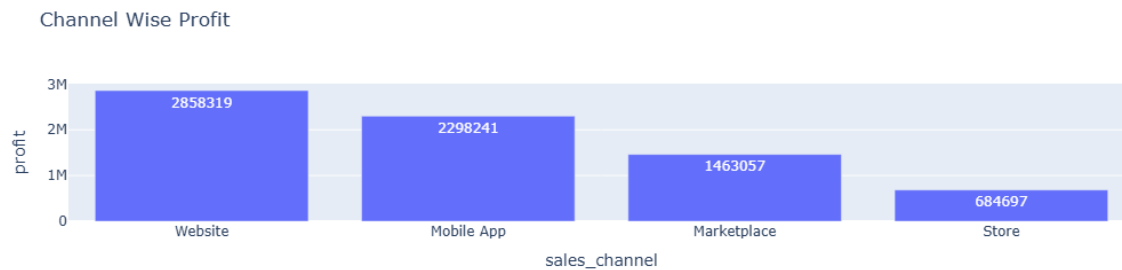
```
print("sales chanel wise Total Profit")
sc_profit = df.groupby("sales_channel",as_index=False)
["profit"].sum().sort_values("profit",ascending = False)
sc_profit
```

sales chanel wise Total Profit

	sales_channel	profit
3	Website	2858319
1	Mobile App	2298241
0	Marketplace	1463057
2	Store	684697

```
fig = px.bar(sc_profit,x="sales_channel",y="profit",title="Channel
Wise Profit",text="profit")
fig.update_layout(
    width=600,
    height=300
)
```

```
)
fig.show()
```



Number of Payment Process

```
x = df["payment_method"].value_counts()
x
```

```
payment_method
UPI                2979
Credit Card       2226
Debit Card        1587
Cash on Delivery   1533
Net Banking       1141
Wallet            534
Name: count, dtype: int64
```

Total Sales Through Payment Method

```
payment_method = df.groupby("payment_method",as_index=False)
["sales"].sum().sort_values(by="sales",ascending=False)
payment_method
```

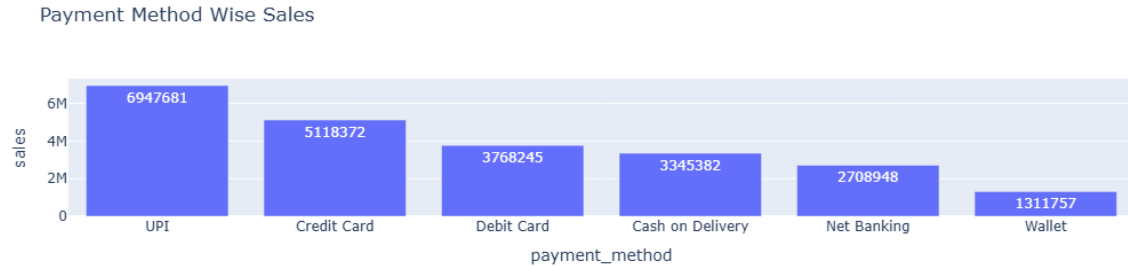
	payment_method	sales
4	UPI	6947681
1	Credit Card	5118372
2	Debit Card	3768245
0	Cash on Delivery	3345382
3	Net Banking	2708948
5	Wallet	1311757

```
fig =
px.bar(payment_method,x="payment_method",y="sales",title="Payment
Method Wise Sales",text="sales")
fig.update_layout(
    width=800,
```

```

        height=300
    )
fig.show()

```



```

day = df.groupby("day_name",as_index=False)
["sales"].sum().sort_values(by="sales",ascending = False)
day

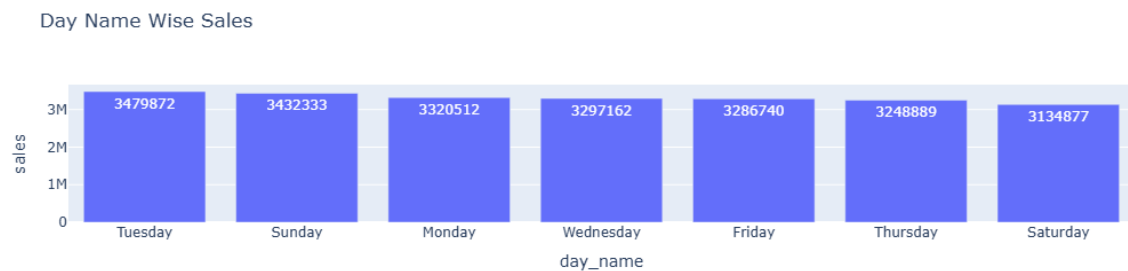
```

	day_name	sales
5	Tuesday	3479872
3	Sunday	3432333
1	Monday	3320512
6	Wednesday	3297162
0	Friday	3286740
4	Thursday	3248889
2	Saturday	3134877

```

fig = px.bar(day,x="day_name",y="sales",title="Day Name Wise
Sales",text="sales")
fig.update_layout(
    width=800,
    height=300
)
fig.show()

```



Month Wise Sales

```
month_order = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"
]

df["month_name"] = pd.Categorical(
    df["month_name"],
    categories=month_order,
    ordered=True
)

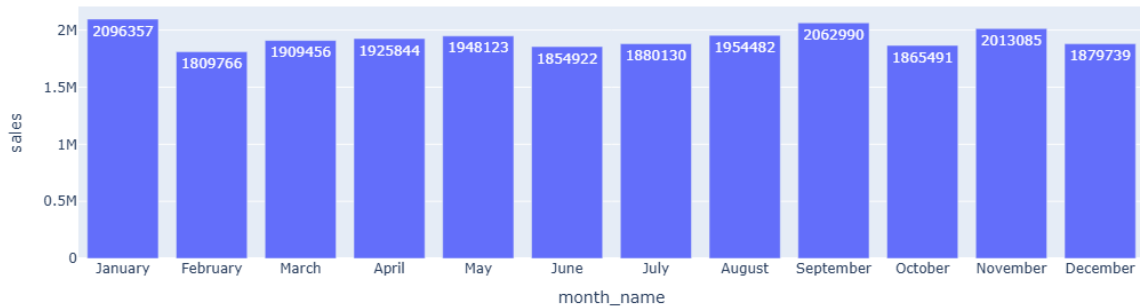
month_Sales = (
    df.groupby("month_name", as_index=False, observed=True)["sales"]
    .sum()
    .sort_values("month_name")
)

month_Sales
```

	month_name	sales
0	January	2096357
1	February	1809766
2	March	1909456
3	April	1925844
4	May	1948123
5	June	1854922
6	July	1880130
7	August	1954482
8	September	2062990
9	October	1865491
10	November	2013085
11	December	1879739

```
fig = px.bar(month_Sales, x="month_name", y="sales", title="Month Name
Wise Sales", text="sales")
fig.update_layout(
    width=800,
    height=400
)
fig.show()
```

Month Name Wise Sales



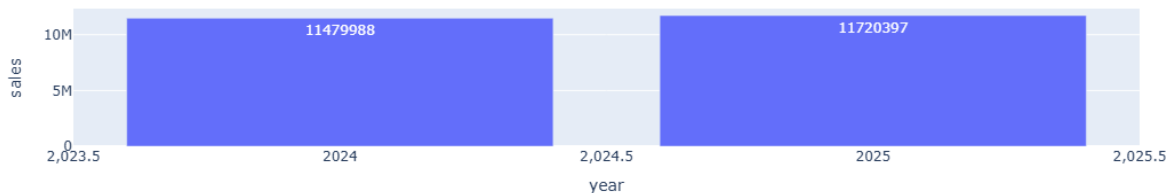
Year wise Sales

```
year_sales = df.groupby("year",as_index=False)
["sales"].sum().sort_values(by="sales",ascending = True)
year_sales
```

	year	sales
0	2024	11479988
1	2025	11720397

```
fig = px.bar(year_sales,x="year",y="sales",title="Year Wise
Sales",text="sales")
fig.update_layout(
    width=800,
    height=300
)
fig.show()
```

Year Wise Sales



Maharashtra Sales and Category

```
maharashtra_category = (
    df[df["state"] == "Maharashtra"]
```

```
.groupby("category", as_index=False)["sales"]  
.sum()  
.sort_values("sales", ascending=False)  
)
```

maharashtra_category

	category	sales
2	Clothing	918675
3	Electronics	881989
5	Home & Kitchen	748090
4	Furniture	618509
1	Books	498473
0	Beauty	439761
6	Sports	425714